Deep Learning for Portfolio Optimization

Group No. 8

- **Rajeshwari – A033**
- **Shivam – A034**
- **Sarrah – A044**
- **Jigar – A046**
- **Tejas – A059**

1. Introduction and Objective

Portfolio optimisation means choosing how to split money across assets so that return per unit of risk is as high as possible. The classical route first forecasts expected returns and then optimises weights. Our reference paper (Zhang, Zohren and Roberts) skips the forecasting step: a neural network reads recent market data and outputs portfolio weights directly, and it is trained to maximise the portfolio **Sharpe ratio** (return divided by risk). We first reproduce that model, which uses an LSTM, and then ask whether two ideas from other papers improve it.

- **Attention** as an alternative to the LSTM for reading the price history (Vaswani et al.; Esslinger et al.).
- **EIIE** as an alternative way of wiring the network: one small shared network per asset instead of one large network for all assets (Jiang et al.).

The project runs in three stages: **(0) reproduction** of the LSTM model; **(1) Part 1**, a comparison of four model combinations under identical rules; and **(2) Part 2**, applying the best combination to Indian ETFs. The four combinations form a 2 x 2 grid, which makes the comparison easy to read:

	Integrated wiring (all assets in one network)	EIIE wiring (one shared mini-network per asset)
LSTM encoder	A. LSTM (reproduction baseline)	C. LSTM + EIIE
Attention encoder	B. Attention	D. Attention + EIIE

2. Building Blocks from the Reference Papers

2.1 Direct Sharpe optimisation (Zhang, Zohren and Roberts)

The network maps recent market data to portfolio weights. The weights are passed through a softmax so that they are positive and sum to one (a long-only portfolio). The training signal is the Sharpe ratio of the resulting portfolio returns, so the network is optimised for the quantity that investors actually care about, instead of a forecasting error that may not translate into a better portfolio. The paper trades four ETFs that represent stocks, bonds, commodities and volatility, because such indices are generally weakly or negatively correlated and therefore diversify well. Among the fully connected, convolutional and LSTM networks it tried, the LSTM worked best on daily data, which is why it is our baseline.

2.2 EIIE: Ensemble of Identical Independent Evaluators (Jiang, Xu and Liang)

An EIIE gives every asset its own small evaluator network (a mini-machine). All evaluators are copies with the same parameters. Each one looks only at the history of its own asset and produces a score for its growth potential. The evaluators interact at only one point, the final softmax, where the scores become portfolio weights. Jiang et al. report three benefits: training cost that grows roughly linearly with the number of assets, better use of data (each stretch of history trains the same evaluator once per asset), and the freedom to change the asset list without retraining from scratch. They also found that an integrated network tends to memorise the past performance of individual assets and then avoids a historically poor asset even when it starts to recover. Because an evaluator does not know which asset it is looking at, it must judge from recent behaviour.

2.3 Attention (Vaswani et al.; Esslinger et al.)

An LSTM reads a sequence one step at a time and carries a memory forward. Self-attention instead lets every position in the sequence look directly at every other position and assign weights to the ones that matter, however far back they are. Its core operation, scaled dot-product attention, is

$$\text{Attention}(Q, K, V) = \text{softmax}(Q K^{T} / \sqrt{d_k}) V \quad (1)$$

where the queries Q, keys K and values V are learned linear projections of the input sequence. Multi-head attention runs several of these in parallel so that different heads can focus on different patterns. Because attention has no built-in notion of order, a positional encoding is added to each input. In the original paper a learned positional encoding gave nearly the same results as a fixed sinusoidal one. Attention connects any two positions in a constant number of steps, whereas a recurrent network needs a number of steps proportional to the distance.

The DTQN paper (Esslinger et al.) uses this idea on a fixed-length history window of the last 50 observations: each observation is linearly embedded, a learned positional encoding is added, and stacked attention blocks (multi-head attention, feed-forward layer, residual connections and layer normalisation) produce the output. Its results are on reinforcement-learning tasks, not finance, so we use it as design guidance only. One useful lesson: its simple attention baseline without layer normalisation, skip connections or stacked blocks learned quickly but was unstable, so we use the full block.

2.4 What we take from each paper

Paper	We take	We do not use
Zhang et al.	Data, Sharpe objective, softmax output, LSTM (64 units), training and evaluation protocol	Fully connected and CNN variants
Jiang et al.	The EIIE wiring: shared per-asset evaluators joined at a softmax	Portfolio-vector memory (PVM), cash asset, reinforcement-learning framing
Vaswani et al.	Multi-head self-attention, positional encoding, residual + layer-norm blocks	Encoder-decoder translation setup
Esslinger et al.	Attention over a 50-step window, learned positional encoding, 2 layers / 8 heads / width 128 as a starting point	Q-learning loss, replay buffer, causal masking

3. Common Framework Shared by All Four Methods

Everything except the model itself is identical for A, B, C and D. This is what makes the comparison fair. Figure 1 shows the whole pipeline; only step 3 changes between methods.

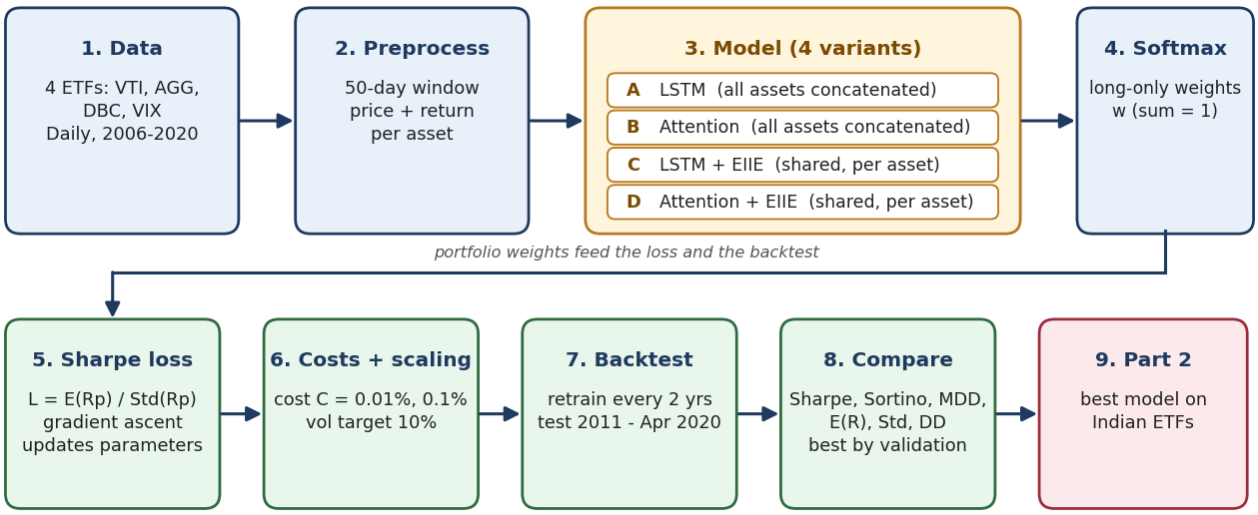


Figure 1. End-to-end pipeline. Steps 1, 2 and 4 to 8 are the same for all four models.

3.1 Data and features

- **Assets:** VTI (US total stock market), AGG (US aggregate bonds), DBC (commodities) and VIX (volatility), daily observations from 2006 to 2020.
- **Features per asset per day:** closing price and daily return. Returns can be derived from prices, but keeping them explicitly helps the objective and acts as a momentum feature.
- **Window:** the past 50 days. For the integrated methods this gives a 50 x 8 input (4 assets x 2 features). For the EIIE methods each asset gets its own 50 x 2 input.

3.2 Model output and training objective

For each sample the network produces raw scores, which a softmax turns into long-only weights:

$$w_{i,t} = \exp(\tilde{w}_{i,t}) \,/\, \sum_j \exp(\tilde{w}_{j,t}) \tag{2}$$

The weights chosen with information up to day t-1 are applied to the returns of day t, so there is no look-ahead. The portfolio return, and the Sharpe ratio over a trading period of T days, are

$$R_{p,t} = \sum_i w_{i,t-1} \cdot r_{i,t} \tag{3}$$

$$L_T = E(R_{p,t}) \,/\, \sqrt{(E(R_{p,t}^2) - (E(R_{p,t}))^2)} \tag{4}$$

The parameters θ of the network are updated by gradient ascent on this Sharpe ratio, where α is the learning rate (we use the Adam optimiser):

$$\theta_{new} = \theta_{old} + \alpha \cdot \partial L_T / \partial \theta \tag{5}$$

3.3 Training procedure (repeated for every model)

1. Take all data available up to the current retraining date; hold out 10% of it as a validation set.
2. Build 50-day windows and, for each window, compute the weights with the model.
3. Compute portfolio returns (Eq. 3) and the Sharpe ratio (Eq. 4) over the mini-batch of 64.
4. Update the parameters by gradient ascent (Eq. 5); train for 100 epochs and tune hyperparameters on the validation set only.
5. Freeze the model and use it to produce daily weights for the next 2 years (the test window); then retrain with the newly available data and repeat, from 2011 to the end of April 2020.

3.4 Evaluation-time adjustments: volatility scaling and costs

When results are reported, positions are scaled so that each asset targets the same volatility, and a transaction cost is charged on every change in position. Let σ_{tgt} be the volatility target (10%), $\sigma_{i,t-1}$ an exponentially weighted 50-day volatility estimate of asset i, and C the cost rate. The scaled position and the reported return are

$$u_{i,t-1} = (\sigma_{tgt} \,/\, \sigma_{i,t-1}) \cdot w_{i,t-1} \tag{6}$$

$$R_{p,t} = \sum_i u_{i,t-1} \cdot r_{i,t} - C \cdot \sum_i |\, u_{i,t-1} - u_{i,t-2} \,| \tag{7}$$

We report results without scaling, with scaling at C = 0.01%, and with scaling at the higher cost C = 0.1%. Volatility scaling makes strategies comparable, because their risk levels are then equal.

4. The Four Methods in Detail

The methods differ in two independent choices: how the price history is read (LSTM or attention) and how the assets are wired (one integrated network, or one shared evaluator per asset). Figures 2 and 3 illustrate these two choices; the cards that follow describe each method.

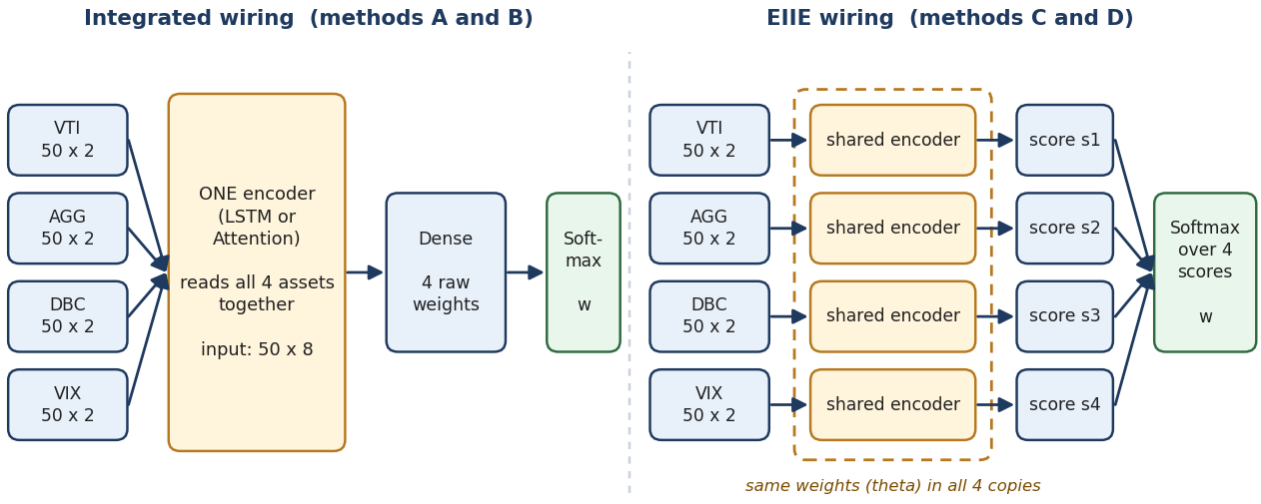


Figure 2. Integrated wiring (A, B) versus EIIE wiring (C, D). Inputs are 50 days x 2 features per asset.

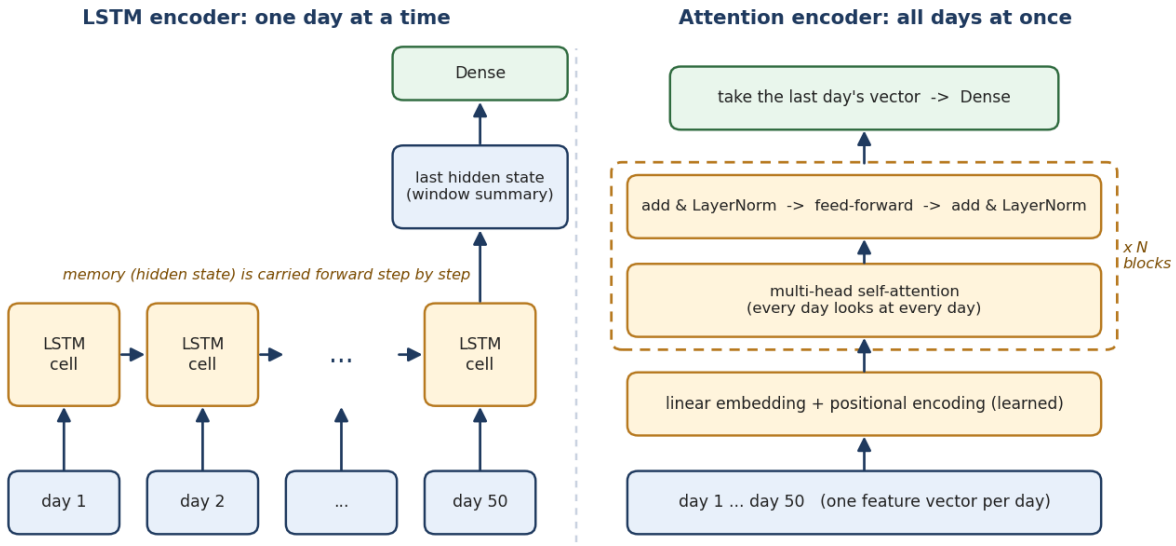


Figure 3. The two encoders. The LSTM reads day by day; attention lets all days interact at once.

Method A: LSTM (integrated), the reproduction baseline	
Idea	One LSTM reads the last 50 days of all four assets together and decides all four weights jointly.
Input	A 50 x 8 matrix per sample: 50 days, and per day the price and return of each of the 4 assets side by side.
Processing	A single-layer LSTM with 64 units reads the days in order and carries a memory forward. Its final hidden state summarises the window, and a dense layer maps it to 4 raw weights.
Output	Softmax over the 4 raw weights gives long-only portfolio weights.
Why	Zhang et al. found the LSTM best on daily data: fewer parameters than a fully connected network (less overfitting) and no over-smoothing of the inputs as in a CNN. Because it sees all assets at once, it can use cross-asset (cross-sectional) information.
Watch out	It must reproduce Zhang's Table 1 before anything else is built on it. Information from early days must survive many recurrent steps.

Method B: Attention (integrated)	
Idea	Replace the LSTM with self-attention: instead of walking through the window day by day, every day looks at every other day and weights the ones that matter.
Input	The same 50 x 8 matrix. Each day's vector of 8 numbers is one token.
Processing	(1) Linear embedding of each day to width 128. (2) Add a learned positional encoding so the model knows the order of days. (3) Two blocks of multi-head self-attention (8 heads), each followed by add & layer-norm, feed-forward layer and add & layer-norm. (4) Take the vector of the most recent day and map it with a dense layer to 4 raw weights.
Output	Softmax over the 4 raw weights.
Why	Any two days are connected directly, so a relevant day far back is as reachable as yesterday. Training is parallel across days. DTQN found a transformer faster and more stable than an LSTM in its (non-financial) tasks, which motivates the test.
Watch out	More parameters than the LSTM on only daily data, so overfitting is the main risk; we use dropout (reported to help by Vaswani et al.), a small width and validation. No causal mask is needed because we read only the last position and all 50 days are already known.

Method C: LSTM + EIIE	
Idea	Give each asset its own copy of the LSTM. Every copy sees only its own asset and has the same weights θ; each outputs one score, and the scores are compared in the softmax.
Input	For each asset i, its own 50 x 2 matrix (price and return). Four such inputs per sample.
Processing	The same 64-unit LSTM runs on each asset's matrix; a shared dense layer turns its last hidden state into one score s_i. The four scores are stacked.
Output	Softmax over the 4 scores gives the weights (Figure 2, right).
Why	Following Jiang et al., one evaluator trained on all assets learns what a rising or falling asset looks like instead of remembering which asset is which, and each stretch of history trains it four times.
Watch out	The evaluators cannot see each other, so the cross-asset information that method A uses is lost. VIX, bonds and stocks behave very differently, so one shared evaluator may fit them poorly. With only 4 assets, EIIE's scalability benefit is small. Comparing C with A tests exactly this.

Method D: Attention + EIIE	
Idea	Combine both ideas: the shared per-asset evaluator of method C, but with the attention encoder of method B inside it.
Input	Each asset's own 50 x 2 matrix; each day's 2 numbers are one token.
Processing	The same attention encoder as method B (embedding, positional encoding, 2 blocks, 8 heads) is applied to each asset with shared weights. The last day's vector goes through a shared dense layer to one score per asset.
Output	Softmax over the 4 scores.
Why	Attention picks out the relevant days in an asset's own history, while EIIE shares that skill across all assets.
Watch out	Attention here runs only over time within one asset. It does not model relationships between assets; that would be a different design and is outside this plan. Keep the model width modest, because each token has only 2 inputs.

5. Experimental Design

5.1 What each comparison tells us

- **A vs C and B vs D:** the effect of EIIE wiring, holding the encoder fixed.
- **A vs B and C vs D:** the effect of attention versus LSTM, holding the wiring fixed.
- **A vs D:** the overall effect of using both ideas together.

5.2 Rules that keep the comparison fair

- Identical data, train/validation/test split, features, loss, optimiser and retraining schedule for all four methods.
- Several random seeds per method; report the mean and spread rather than a single run.
- Report the parameter count of each model. The EIIE variants have fewer input weights, so if the gap is large, adjust the width on the validation set and say so.
- Tune only on the validation set. Choose the winner by validation Sharpe, and use the test period once, for final reporting.

5.3 Hyperparameters

Item	Value	Source
LSTM hidden units	64, single layer	Zhang et al.
Attention layers / heads / width	2 / 8 / 128 (starting point, tuned on validation)	Esslinger et al. (Table 2)
Window length	50 days	Zhang et al.; Esslinger et al.
Optimiser, batch, epochs	Adam, 64, 100	Zhang et al.
Validation share	10% of training data	Zhang et al.
Learning rate, dropout	Tuned on validation (not fixed by the papers)	Project choice

6. Evaluation

6. Metrics

Metric	Meaning
E(R), Std(R)	Annualised expected return and its standard deviation (volatility)
Sharpe ratio	Expected return divided by volatility: return per unit of total risk (main comparison metric)
DD(R), Sortino ratio	Downside deviation (volatility of losses only) and expected return divided by it
MDD	Maximum drawdown: the largest fall from a peak to a later trough
% of + Ret, Ave. P / Ave. L	Share of days with a positive return, and average profit divided by average loss

For context we can also quote the paper's baselines (four fixed allocations, mean-variance, maximum diversification and the diversity-weighted portfolio).

7. Transfer to Indian ETFs

Once Part 1 identifies the best of A to D by validation Sharpe, that model is retrained and tested on Indian ETF data with the same protocol: same window, objective, retraining schedule, cost levels and volatility scaling as a first pass. Three things must be settled before training:

1. The exact ETF list, ideally one per asset class (equity, bonds, commodities and, if available, volatility).
2. Whether a volatility exposure is available as an ETF for the Indian market; if not, decide what replaces the fourth asset.
3. Whether the ETF histories are long enough for the 2-year retraining scheme, and what the test period will be.

8. References

#	Paper and link	Used for
1	**Zhang, Zohren & Roberts (2020). Deep Learning for Portfolio Optimization** arxiv.org/abs/2005.13665	Main paper; reproduction target
2	**Jiang, Xu & Liang (2017). Deep Portfolio Management** arxiv.org/abs/1706.10059	EIIE wiring
3	**Vaswani et al. (2017). Attention Is All You Need** arxiv.org/abs/1706.03762	Attention mechanism
4	**Esslinger, Platt & Amato (2022). Deep Transformer Q-Networks** arxiv.org/abs/2206.01078	Attention over a 50-step window; starting hyperparameters